



Initial System Design & Work Division Report

Honeyops

Team #1

Team Members:

Ali Ahmed Reda

Ali Nazeer

Ahmed Yasser

Abdelikaliq Sarwat

Github link : [Aligithyb/-AdaptiveWardens](https://github.com/Aligithyb/-AdaptiveWardens)

Supervisor: Dr. Ashraf Badawi

Fall 2025

Table of Contents

1. Project Summary	1
2. Progress Since Proposal	2
2.1 Completed Tasks 2.2 Research & Design Work	
3. System Architecture	3
4. Component Breakdown	5
5. Data Flow	6
6. Work Breakdown Structure	7
7. Risk Analysis	8

1. Project Summary

Honeyops is an AI-driven, high-interaction honeypot designed to simulate realistic Linux and web environments for cybersecurity research and incident analysis.

The system uses an AI Engine to dynamically generate believable SSH and HTTP responses, maintain a persistent fake OS state, extract Indicators of Compromise (IOCs), and map attacker behavior to the MITRE ATT&CK framework.

A full SOC-style dashboard provides real-time monitoring, session playback, IOC insights, and attacker behavior analytics.

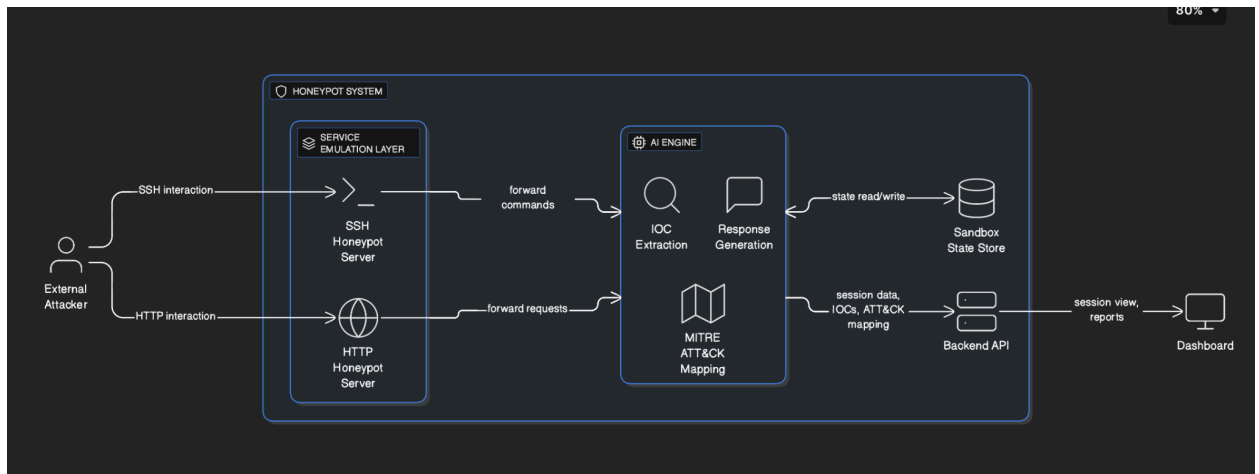
The project consists of five major components: SSH Frontend, HTTP Frontend, AI Engine, Sandbox/State Store and Dashboard — all combining to create an adaptive, deception-based cybersecurity system.

2. Progress Since Proposal

Since Week 4, our main progress has focused on designing the website mockup and testing multiple existing honeypots to inform our system's architecture and deception strategy. The website mockup defines the layout and structure of the SOC dashboard—including Live Sessions, Session Playback, IOC views, and the ATT&CK heatmap—ensuring a clear visual plan for the final frontend. In parallel, we tested honeypots such as Cowrie, Dionaea, and OpenCanary, evaluating their strengths, limitations, and attacker interaction patterns. This research directly influenced our system's feature design, realism goals, and the improvements we aim to introduce with an adaptive AI-driven honeypot.

3. System Architecture

Fig. 1



4. Component Breakdown

SSH Frontend

Function

Acts as the fake SSH server that accepts attacker connections, captures activity, and relays commands to the AI Engine.

Inputs

- Raw SSH connections
- Credentials (valid/invalid)
- Commands entered by attacker
- Timing data (keystrokes, latency)

Expected Outputs

- AI-generated terminal output
- Logged session activity

- Streamed events to dashboard backend

Main Functionality

- Simulates a real Linux SSH environment
- Captures:
 - commands
 - timestamps
 - session metadata
 - connection IP/User-Agent (if available)
- Sends commands to the AI Engine
- Outputs generated responses back to attacker
- Maintains realistic terminal behavior (echo, delays, pseudo-TTY)
- Forwards full logs to Dashboard Backend

Technologies

- Python
- AsyncSSH or Paramiko
- FastAPI (for log submission to backend)

HTTP Frontend

Function

A deception HTTP server that accepts any type of web request and dynamically generates believable responses using the AI Engine.

Inputs

- GET/POST requests
- Request headers
- Request body (including payloads, scans, fuzzing input)
- Route/endpoint requested

Expected Outputs

- AI-generated HTML/JSON/text responses

- Logged request & payload data
- Extracted IOCs sent to backend

Main Functionality

- Accepts arbitrary routes (including non-existing ones)
- Identifies scanner behavior
- Forwards request metadata + payload to AI Engine
- Returns realistic web-server output (status codes, HTML pages, JSON APIs)
- Logs all activity for dashboard and analysis

Technologies

- FastAPI
- Uvicorn/Gunicorn
- Python Requests/HTTPX for internal communication

AI Engine (Core Brain)

Function

Generates dynamic fake system behavior, manages sandbox state, extracts intelligence, and maps attacker activity to MITRE ATT&CK.

Inputs

- Command from SSH frontend
- HTTP request data
- Current sandbox state (filesystem, processes, logs, history)
- Session history

Expected Outputs

- Generated command output / webpage content
- Updated sandbox state
- Extracted IOCs (IPs, domains, hashes, URLs, filenames)
- ATT&CK technique mappings
- Session summaries for dashboard

Main Functionality

- Generate realistic Linux terminal output
- Generate realistic HTTP responses
- Mutate sandbox state:
 - filesystem writes
 - process creation
 - log generation
 - history updates
- IOC extraction using NER models
- Technique detection → MITRE ATT&CK mapping
- Packet logs → structured JSON for dashboard backend

Technologies

- Python
- PyTorch / TensorFlow
- HuggingFace Transformers
- SQLite / JSON for state writes

Sandbox / State Store

Function

A fully simulated OS environment where all fake system data lives. Ensures persistence and realism with NO real OS risk.

Inputs

- State mutations from AI Engine
- Session events (commands, file writes, processes)
- Log entries

Expected Outputs

- Updated system state:
 - filesystem tree
 - logs
 - running processes
 - history
- Persistent per-session “fake OS”
- State snapshots accessible by dashboard

Main Functionality

- Fake filesystem (read/write)
- Fake process table
- Fake log files (/var/log/auth.log, syslog)
- Per-session bash history
- Safe environment that never executes actual commands
- Readable/writable via API internal calls

Technologies

- SQLite (tables for FS, processes, logs, history)
- JSON (cached copies or fast read/write structures)
- Python for state management layer

Dashboard System (Backend + Frontend)

Function

Serves as the complete SOC-facing intelligence and monitoring layer. The backend collects, stores, and serves all honeypot data, while the frontend visualizes attacker behavior through real-time dashboards, playback tools, and analytical views.

Inputs

- Logs from SSH and HTTP frontends
- IOC data extracted by the AI Engine

- MITRE ATT&CK technique mappings
- Session metadata and attacker activity timelines
- Sandbox snapshots (filesystem, processes, logs)
- API endpoints (used by frontend to retrieve backend data)

Expected Outputs

- REST API responses delivering sessions, logs, and metrics
- WebSocket real-time streams for live activity
- IOC tables with filters and enrichment data
- MITRE ATT&CK heatmap visualizations
- Session playback timeline and analytics
- Charts, graphs, and comparison metrics (AI honeypot vs traditional honeypots)

Main Functionality

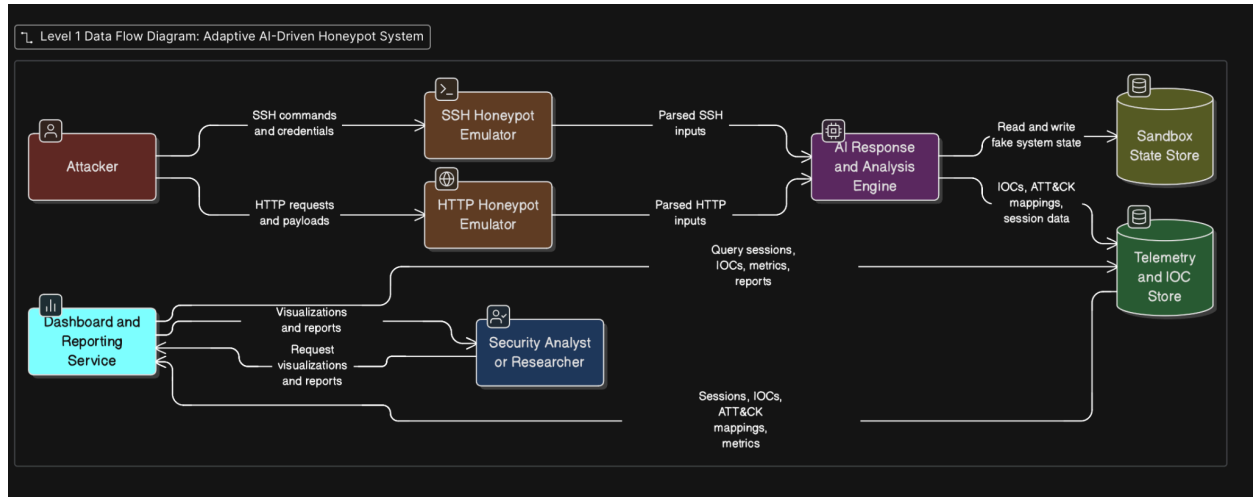
- Aggregate and index all honeypot data for fast querying
- Provide filtering by time, IP, IOC, tool, or technique
- Store full attacker sessions with timestamps for replay
- Stream live attacker activity to the dashboard in real time
- Display active attackers, commands, HTTP requests, and IOCs
- Replay full attack sessions with a timeline slider
- Visualize MITRE technique usage and attacker patterns
- Display metrics such as commands/session, session length, engagement level
- Support exporting data (CSV/JSON) for reporting and analysis

Technologies

- Backend: FastAPI, Python, SQLite/PostgreSQL, etc..
- Frontend: React, TypeScript, TailwindCSS, Recharts , etc..

5. Data Flow

Fig. 2



6. Work Breakdown Structure

WB S ID	Task Name	Detailed Technical Description	Owner	Start Date	End Date	Status
1.0	Attacker Interaction Capture	Implement SSH & HTTP listeners; capture raw attacker commands/requests; forward input to backend API.	Ali Nazeer	10 Feb	14 Feb	Planned
1.1	SSH Input Pipeline	Build AsyncSSH handler, session tracking, input normalization.	Ali Nazeer	10 Feb	12 Feb	Planned

1.2	HTTP Input Pipeline	Create FastAPI route for HTTP honeypot endpoints; parse headers and path.	Ali Nazeer	12 Feb	14 Feb	Planned
2.0	AI Prompt Construction	Build state snapshot generator, history bundler, and instruction block assembler.	Ali Ahmed	11 Feb	16 Feb	Planned
2.1	State Snapshot Serialization	Convert fake filesystem, processes, logs, env vars → JSON snapshot.	Ali Ahmed	11 Feb	13 Feb	Planned
2.2	Prompt Builder	Merge state + history + input into LLM-ready format.	Ali Ahmed	13 Feb	16 Feb	Planned
3.0	Tokenization & Embedding Pipeline	Pass prompt through tokenizer (BPE), generate embedding matrices.	Abdulkhali q	12 Feb	17 Feb	Planned
3.1	BPE Tokenizer Integration	Integrate HF tokenizer; convert text → token IDs.	Abdulkhali q	12 Feb	14 Feb	Planned
3.2	Embedding Stage	Ensure embeddings load and feed into transformer correctly.	Abdulkhali q	14 Feb	17 Feb	Planned
4.0	Transformer Reasoning Process	Configure inference pipeline: attention, feed-forward, heads.	Ahmed Yasser	13 Feb	20 Feb	Planned
4.1	Self-Attention Optimization	Validate attention weights interpret state/history correctly.	Ahmed Yasser	13 Feb	18 Feb	Planned

4.2	Multi-Head Output Handling	Enable LM head (OS output) + Instruction head (JSON actions).	Ahmed Yasser	18 Feb	20 Feb	Planned
5.0	Output Generation	Produce terminal-like OS output + sandbox mutation actions.	Ali Ahmed	16 Feb	19 Feb	Planned
5.1	OS Output Generation	Generate human-readable shell/HTTP responses.	Ali Ahmed	16 Feb	18 Feb	Planned
5.2	JSON Mutation Output	Produce validated actions (write_file, read_file, spawn_process).	Ali Ahmed	18 Feb	19 Feb	Planned
6.0	Safety Validation Layer	Validate JSON actions, block harmful or inconsistent mutations.	Ali Nazeer	15 Feb	18 Feb	Planned
6.1	Mutation Parser	Parse & check JSON mutation packets.	Ali Nazeer	15 Mar	17 Mar	Planned
6.2	Safety Rules Engine	Ensure disallowed operations are prevented.	Ali Nazeer	17 Mar	18 Mar	Planned
7.0	Sandbox Update Engine	Update fake filesystem, processes, logs based on validated actions.	Abdulkhaliq	17 Mar	21 Mar	Planned
7.1	Filesystem Mutation Logic	Apply write/delete/read changes to SQLite/store.	Abdulkhaliq	17 Mar	19 Mar	Planned

7.2	Process & Log Simulation	Generate fake PIDs, append logs, update timestamps.	Abdulkhaliq	19 Mar	21 Mar	Planned
8.0	IOC Extraction + MITRE Mapping	Extract indicators and map behaviors to ATT&CK techniques.	Ahmed Yasser	20 Mar	24 Mar	Planned
8.1	IOC Extraction Module	Detect IPs, URLs, hashes, toolnames.	Ahmed Yasser	20 Mar	22 Mar	Planned
8.2	MITRE Technique Classifier	Map behaviors to TTPs (T1059, T1105, etc).	Ahmed Yasser	22 Mar	24 Mar	Planned
9.0	Dashboard Integration	Send outputs + telemetry to dashboard in real time.	Team	24 Mar	28 Mar	Planned

7. Risk Analysis

1.1 Technical Risks:

T1: AI Model Performance & Response Quality (Impact: High, Probability:

Medium-High, Priority: CRITICAL)– The transformer model may generate unrealistic responses that attackers can easily identify. Mitigation: Test with real command datasets, implement quality validation layers, use pre-validated responses as fallback, conduct red team simulations, and apply timing variations.

T2: State Consistency Issues (Impact: High, Probability: Medium, Priority: HIGH)–

The sandbox state store may become inconsistent across sessions. Mitigation: Implement state validation logic, create rollback mechanisms, design comprehensive unit tests, use SQLite transactions for checkpointing.

T3: Integration Complexity (Impact: High, Probability: Medium-High, Priority: HIGH)– Component communication issues between SSH/HTTP frontends, AI engine, sandbox, and dashboard. Mitigation: Define clear API contracts using OpenAPI, implement integration tests from week 1, use weekly integration sprints, create mock services for parallel development.

T4: Performance Bottlenecks (Impact: Medium, Probability: Medium)– AI inference latency may cause unrealistic response times. Mitigation: Early profiling and benchmarking, implement response caching, use FastAPI async processing, apply model quantization, conduct load testing..

T5: Security Vulnerabilities (Impact: Critical, Probability: Low-Medium, Priority: CRITICAL)– Attackers could exploit vulnerabilities to compromise the host system. Mitigation: Never execute real commands, run in isolated containers with minimal privileges, strict input validation, regular security code reviews, use established libraries like Paramiko.

1.2 Resource Risks

R1: GPU/ComputeResourceAvailability(Impact: Medium, Probability: Medium)–

Insufficient GPU resources or budget constraints. Mitigation: Use university GPU clusters or free-tier cloud services (Google Colab, Azure for Students), select lightweight transformer models (DistilBERT, BERT-tiny), optimize for CPU execution, implement caching.

R2: Team Member Availability (Impact: Medium-High, Probability: Medium)–

Conflicting academic schedules or personal commitments. Mitigation: Build buffer time into a 25-week timeline, cross-train team members, document thoroughly, front-load critical items, maintain backup communication channels .

R3: Data/Dataset Availability (Impact: Medium, Probability: Medium)– In sufficient

real attacker data for training and evaluation. **Mitigation:** Use public honeypot datasets (Cowrie logs, HoneyDB), generate synthetic scenarios based on MITRE ATT&CK, deploy basic honeypot early for data collection, use CTF scenarios.

1.3 Timeline Risks

TL1: Scope Creep (Impact: High, Probability: Medium-High, Priority: HIGH) Adding features beyond original scope may derail timeline. Mitigation: Strictly adhere to defined scope boundaries, maintain feature backlog for post-graduation, require supervisor approval for changes, deliver MVP first, conduct bi-weekly scope reviews .

TL2: Implementation Phase Delays (Impact: High, Probability: Medium, Priority:

HIGH)– Development (weeks 9-17) may take longer than estimated. Mitigation: Prioritize core features first, use modular development for parallel work, set 2-3 week milestone checkpoints, have backup simplified implementations, start testing by week 10.